

**Desarrollar la arquitectura de software de acuerdo al patrón de diseño  
seleccionado GA4-220501095-AA2-EV05**

**APRENDIZ**  
**Johan Mauricio Sánchez Gaviria**

**Análisis y Desarrollo de Software**

**FICHA**  
**3134555**

**INSTRUCTOR**  
**Nelson Guzman**

## **INTRODUCCION**

Los patrones de diseño son herramientas esenciales en el desarrollo de software, proporcionando soluciones probadas para estructurar la interacción entre objetos, clases e interfaces. En el transcurso de este documento, exploraremos las diversas estructuras que los patrones de diseño ofrecen, desglosando cómo estos modelos establecen pautas para la organización y la relación entre los componentes del software.

## DISEÑOS DE PATRONES DE SOFTWARE

Los patrones de diseño representan un concepto originado en el ámbito de la arquitectura de software, extendiéndose gradualmente al diseño orientado a objetos, ya que facilita la reutilización de diseños y arquitecturas de software.

Los patrones de diseño de software es un conjunto de formas estandarizadas para resolver un problema en particular.

Debe estar estructurado por 4 componentes:

1.**Nombre:** Identifica de manera única el patrón, brindando una referencia clara para su aplicación.

2.**Problemas o contexto de la aplicación:** Describe el escenario o los desafíos específicos a los que se enfrenta el patrón. Proporciona un contexto para comprender cuándo y por qué aplicar el patrón.

3.**Solución propuesta:** Detalla la estructura y las interacciones entre los elementos del diseño que componen la solución. Este componente ofrece una guía para implementar el patrón de manera efectiva.

4.**Ventajas y desventajas:** Evalúa los beneficios y las limitaciones asociadas con la aplicación del patrón. Proporciona una visión equilibrada para ayudar a los desarrolladores a tomar decisiones informadas sobre su implementación.

Los patrones de diseño más representativo de la industria del software

- Implica conocimiento y la aplicación de criterio que aportan calidad.
- Favorecen la estandarización del código fuente de determinada aplicación.

Los patrones de diseño se clasifican en 3 grupos.

- Patrones comportamentales
- Patrones creacionales
- Patrones estructurales

**Patrones comportamentales:** Se centran en definir la manera en que los objetos interactúan entre ellos por medio de mensajes.

Se cuentan con 3 tipos:

Estrategia, comando e iterador

**El patrón de Estrategia:** permite definir una familia de algoritmos, encapsular cada uno de ellos y hacer que sean intercambiables. Esto permite que el cliente pueda elegir el algoritmo que mejor se adapte a sus necesidades en tiempo de ejecución.

**Comando:** El patrón de Comando encapsula una solicitud como un objeto, permitiendo parametrizar clientes con diferentes solicitudes, encolar solicitudes y realizar operaciones reversibles.

**Iterador:** El patrón de Iterador proporciona una manera de acceder

secuencialmente a los elementos de una colección sin exponer su estructura interna

**Componentes creacionales:** se encargan de definir diferentes tipos de estrategia que son usados para requerir la instancia de un objeto, clase, en particular.

Aquí encontramos:

**Singleton:** Garantiza que una clase tenga solo una instancia y proporciona un punto de acceso global a ella.

**Prototype:** Permite la creación de nuevos objetos copiando un objeto existente, evitando depender de las clases concretas de esos objetos.

**Factory method:** Proporciona una interfaz para crear objetos en una superclase y permite a las subclases alterar el tipo de objetos que se crearan.

**Builder patterns:** Permite la construcción de un objeto paso a paso. Un director coordina el proceso de construcción utilizando un constructor abstracto.

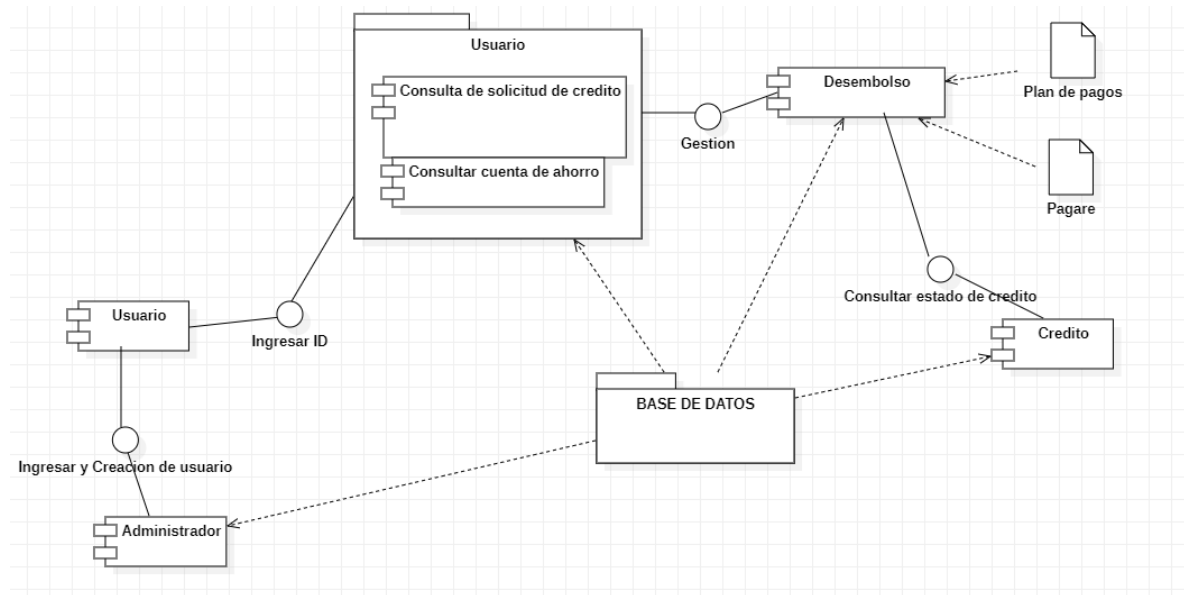
**Abstract factory:** Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.

**Patrones estructurales:** Proveen una orientación relacionada a la forma de definir los componentes de los objetos, dentro de esta encontramos.

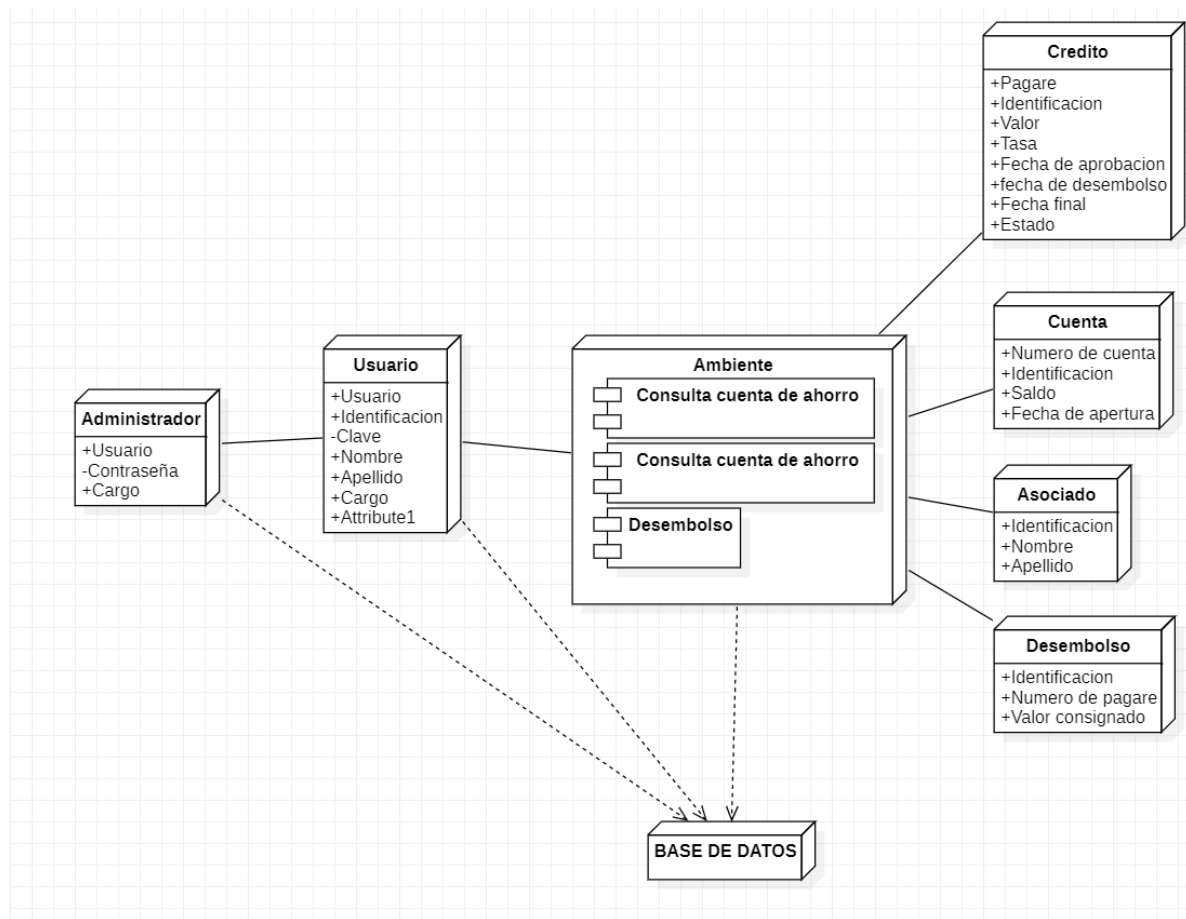
**Patron fachada:** La fachada actúa como una interfaz unificada que proporciona un conjunto simplificado de métodos para interactuar con un sistema más complejo.

**patron delegat:** Implica que un objeto controle la funcionalidad de otro objeto. Actúa como un sustituto o representante de otro objeto y controla el acceso a él.

## DIAGRAMA DE COMPONENTES



## DIAGRAMA DE DESPLIEGUE



## Herramientas necesarias para optimizar los procesos.

Diagrama causa y efecto

Diagrama de flujo

Diagrama de Pareto

Diagrama sipoc

## **CONCLUSIONES**

Los patrones de diseño ofrecen una valiosa guía para resolver problemas de desarrollo al proporcionar soluciones estructuradas. Facilitan la construcción de la interacción entre objetos, clases e interfaces, ayudan a identificar la arquitectura del software y sus relaciones, y se presentan de manera gráfica para una comprensión secuencial efectiva del diseño.